

Journal — Hardik Pajjala (1024030119)

Role: Person A — Frontend & UX (React) Project: FoodBridge — a surplus food redistribution matcher

Week 2 (7 – 13 September 2026)

What I did

Rebuilt the frontend against Shikhar's real API. Shikhar's backend became available this week, and it turned out to implement a different model from the one I had designed against — his lifecycle has a PENDING

REVIEW state and no ACCEPTED donation state, acceptance lives on the match, and pickups are a separate entity. I rewrote the client to match his API rather than asking him to match mine, because his code was implemented and tested while my contract was only a document. About 2,300 lines changed across 30 files. Built the screens his model needed that mine did not have: a login and a registration screen (his API uses JWT, so authentication had to exist), an administrator review queue for approving and rejecting donations, a match inbox replacing my old offer inbox, and a pickups screen. I removed the screens his model made redundant.

Integrated the frontend with the backend and the matching service. Got all three running together and confirmed a donation travels the full path: donor posts → admin approves → matching cycle runs → organisation accepts → pickup → completed.

Wrote a contract-conformance check. It exercises every endpoint the client depends on and verifies status codes, response shapes, the error envelope and the role restrictions, then exits non-zero on failure so it can gate a build. It answers one question: can the React client talk to this server without changes. 23 checks currently pass with no failures.

Tested the running system and found real defects. Working through the app as each role turned up problems that no test suite had caught. These are the ones worth recording: Donations were reaching no organisation at all. The cause was a vocabulary mismatch: my form sent food types like cooked

veg while every organisation listed cooked

meals , and the matcher filters on an exact string, so almost every donation was eligible for nobody and went silently to the waitlist.

The administrator could see that a donation had been matched but not to whom. The backend returns bare UUIDs, so the screen was printing identifiers. It now resolves them into the donation and the organisation's name.

Organisations that looked to have free capacity were being skipped. Unanswered offers reserve capacity for their response window, and because nothing expires them automatically they were holding it indefinitely. The behaviour is correct — refusing to

double-book a shelter — but it degrades allocation quietly, and it is the strongest argument for the scheduled job we still need.

What I learned

Integrating against Shikhar's matching service meant reading it closely, and that made the filter-then-rank design concrete in a way the proposal had not. Eligibility is settled before any scoring, so an organisation that fails a hard constraint never competes on score at all — which is why the nearest organisation is frequently not the one that wins, and why a donation can be matched to an organisation three kilometres away while one at one kilometre is skipped.

I also compared the two allocation strategies on the same scenario to understand what the global assignment actually buys. Given two donations and one nearby organisation with capacity for both, greedy placed both while the Hungarian method placed one and waitlisted the other — because the Hungarian method solves a one-to-one assignment, so an organisation takes at most one donation per cycle. That is worth recording: our proposal frames the global assignment as an upgrade, and on this scenario it distributed less food, so the final evaluation should say which measure each strategy improves rather than presenting one as better outright.

Being able to reason about the matcher from its code rather than from the proposal is what let me diagnose the three defects above. Each of them looked like “matching is broken” from the interface, and none of them was.

Blocked / needs the team

No scheduled job, so nothing expires or re-matches unless an administrator triggers a cycle. Backend side.

/matches/inbox returns no donation detail, so the inbox fetches each donation separately.

One join on Shikhar's side would remove it.

Offer acceptance rate and mean donor-to-recipient distance are named in our evaluation plan but are not in the impact summary, so two of our metrics have no data source.